

Vibration Study of 2.2 Road (Gymkhana to Nehru Museum), IIT Kharagpur

Date: November 12, 2025

Prepared by: C2G15 Group

(P Prajit – 24MF10047, Riddhi Maiti – 24MF10051, Nare Mahith Sai – 24MF10043)

Abstract

This report documents a vibration measurement study carried out along the 2.2 Road route from Gymkhana to Nehru Museum, IIT Kharagpur. Measurements were taken by riding a campus toto equipped with two inertial sensors (MPU6050 and ADXL345) and a GPS receiver to provide GPS tagging. The objective was to characterize road roughness and in-vehicle vibrational environment and provide recommendations for road maintenance and vehicle comfort.

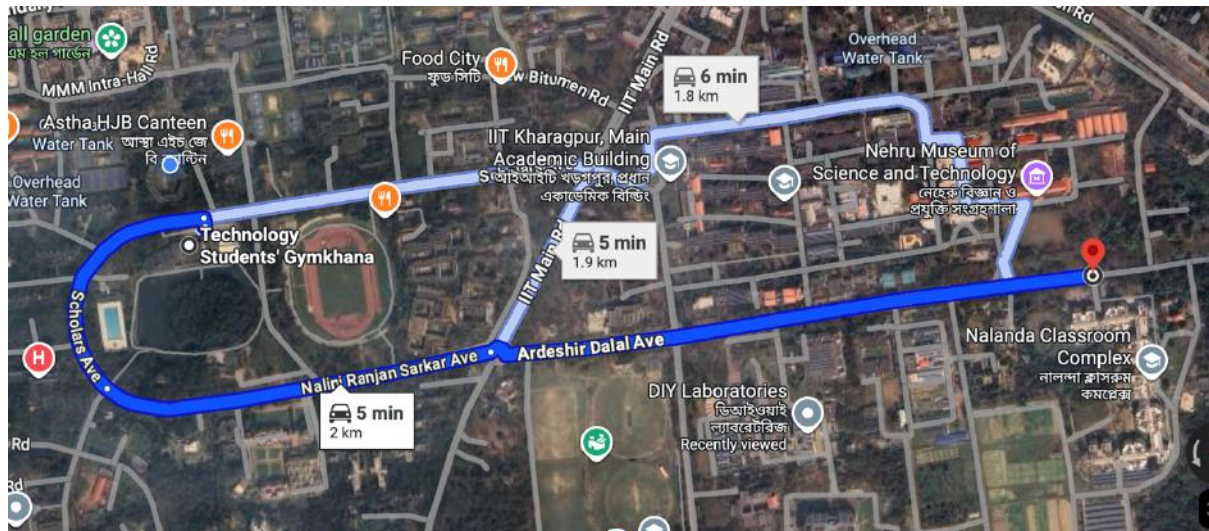
Note : Project partially inspired by Professor Vikranth Racherla's case study on dynamics study of bogeys.

Table of Contents

1. Problem statement
2. State of the art
3. Objectives
4. Work plan and schedule
5. Equipment & specifications
6. Connection diagram
7. Results
8. Discussion
9. Conclusions
10. Recommendations
11. References
12. Appendices
 - A: Arduino code
 - B: Python code for data analysis

1. Problem statement

Road roughness and localized defects cause vehicle vibrations that affect passenger comfort, vehicle fatigue life, and may indicate pavement deterioration. The 2.2 Road between Gymkhana and Nehru Museum at IIT Kharagpur is used frequently by campus vehicles (including totos). The goal of this study is to quantify vibration levels experienced during a typical toto ride along this route, identify hotspots (locations with elevated vibrations), and provide both engineering metrics and actionable recommendations for maintenance.



Key questions:

- Where along the route are the vibration levels unusually high (hotspots)?
- How do the two accelerometers (MPU6050 & ADXL345) compare in measured response?
- What do the vibration metrics imply about passenger comfort and pavement condition?

2. State of the art

Short summary of established practice and literature:

- Road roughness is commonly quantified using profile-derived indices (e.g., International Roughness Index — IRI) or accelerometer-derived metrics (RMS acceleration, power spectral density, Vibration Dose Value — VDV). Accelerometers mounted in vehicles are widely used for quick road condition surveys when profilometers are not available.
- Low-cost MEMS accelerometers (e.g., MPU6050, ADXL345) have been shown in many studies to be adequate for relative road condition mapping when appropriately calibrated and filtered. Typical limitations include bias instability, temperature sensitivity, and reduced accuracy at very low frequencies.
- GPS (NMEA) data are used to geolocate measurement windows and compute speed; careful timestamp synchronization between GPS and IMU data is necessary for reliable hotspot mapping.
- Standard guidance on human exposure to whole-body vibration is available in ISO 2631 series (e.g., ISO 2631-1) which describes metrics such as VDV for comfort evaluation.

3. Objectives

1. To acquire synchronized accelerometer (3-axis) data and GPS position/time data along the entire 2.2 Road route using a toto as the measurement platform.

2. To process raw sensor data to remove noise and bias, align timebases, and transform sensor axes to vehicle axes if required.
3. To compute vibration metrics per road segment: RMS acceleration, peak and VDV (where applicable).
4. To produce a geolocated vibration heatmap showing hotspots along the route and a ranked list of problematic locations.
5. To compare the two accelerometers (MPU6050 vs ADXL345) for consistency and recommend best practices for future surveys.
6. To provide recommendations for road maintenance, ride comfort improvement, and future measurement improvements.

4. Work plan and schedule

Phase 1 — Preparation (1 week)

- To finalize sensor mounting and logging system.
- To write and verify data-logging firmware.
- Pre-test bench calibration.

Phase 2 — Field data collection (1–3 days)

- To perform multiple runs (minimum 2 runs at similar daytime and load) along route at typical toto speeds.
- To record environmental notes (weather, passenger load, route deviations).

Phase 3 — Data processing & analysis (1–2 weeks)

- Preprocess: resample, time-sync, filter, offset removal.
- To compute metrics per fixed-length segments (e.g., 10 m or 50 m windows) and identify hotspots.

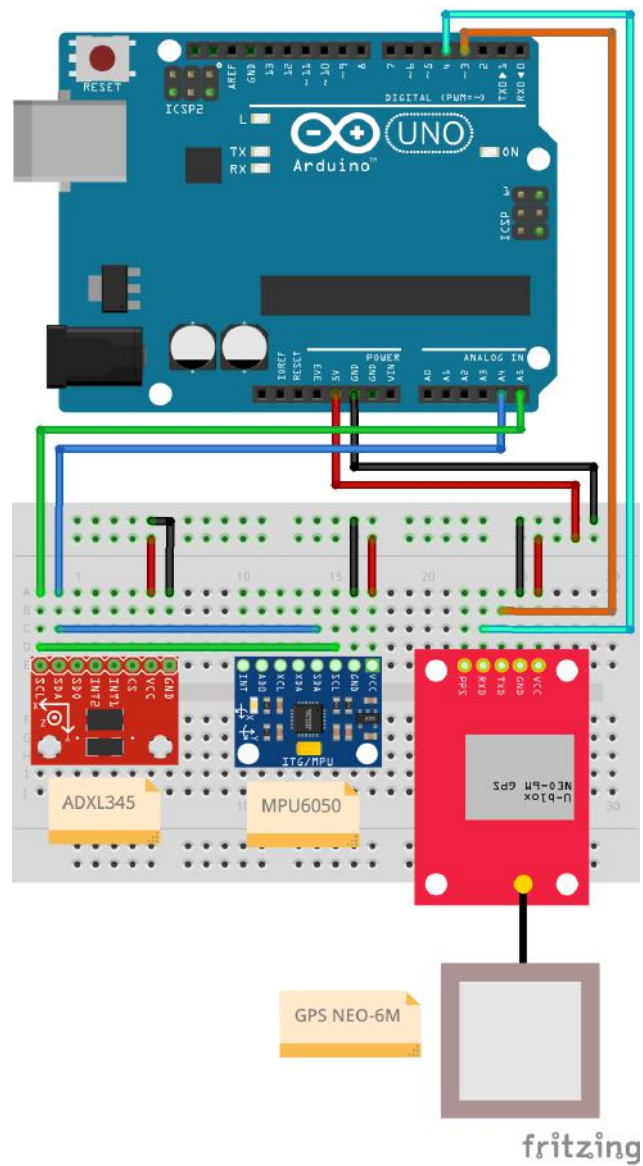
Phase 4 — Reporting & recommendations (1 week)

- To create visualizations, map overlays, and final report.

5. Equipment & specifications

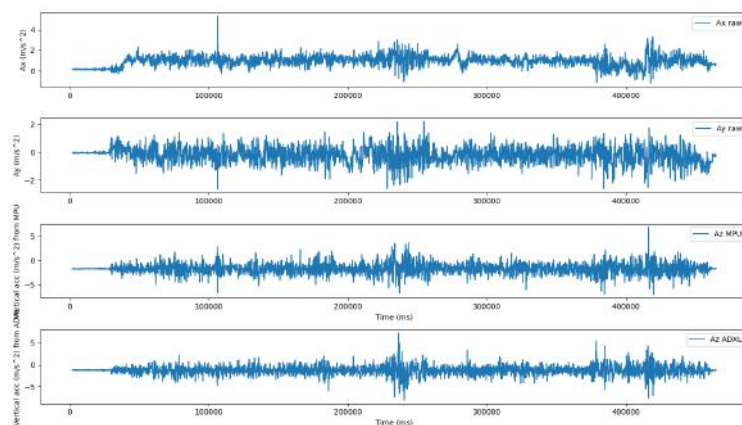
1. **MPU6050** : 3-axis accelerometer + 3-axis gyroscope. Typical accelerometer range $\pm 2/\pm 4/\pm 8/\pm 16$ g (configurable), internal sampling via I2C.
2. **ADXL345** : 3-axis accelerometer, selectable ranges $\pm 2/\pm 4/\pm 8/\pm 16$ g, digital interface (I2C).
3. **GPS module** (Neo-6M) : provides UTC position and time via NMEA sentences; logging at 1–10 Hz.
4. **Arduino Uno** : Microcontroller for data acquisition and timestamps.
5. **Laptop / software**: Python (pandas, numpy, scipy, matplotlib), folium for mapping, Jupyter for processing.

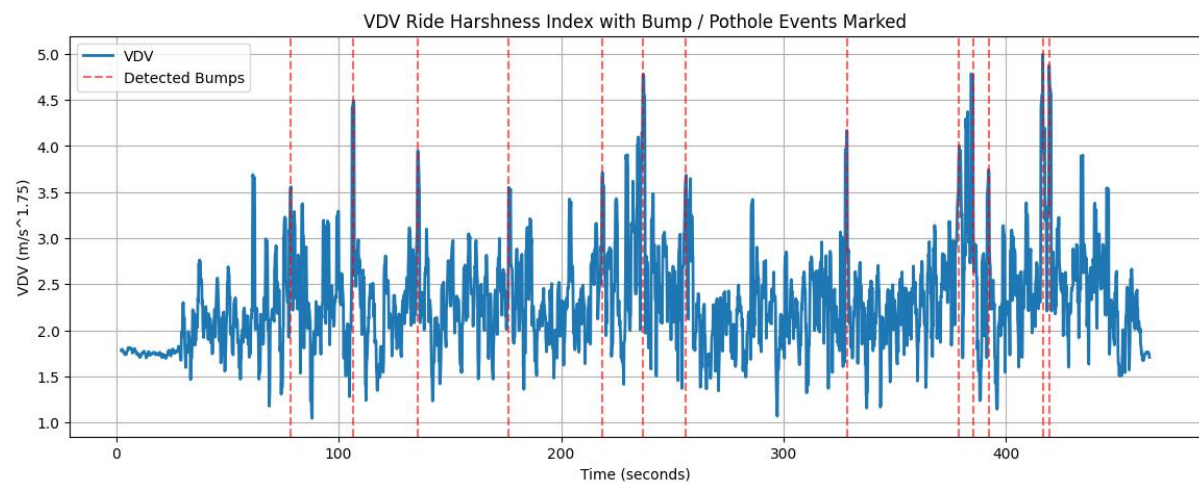
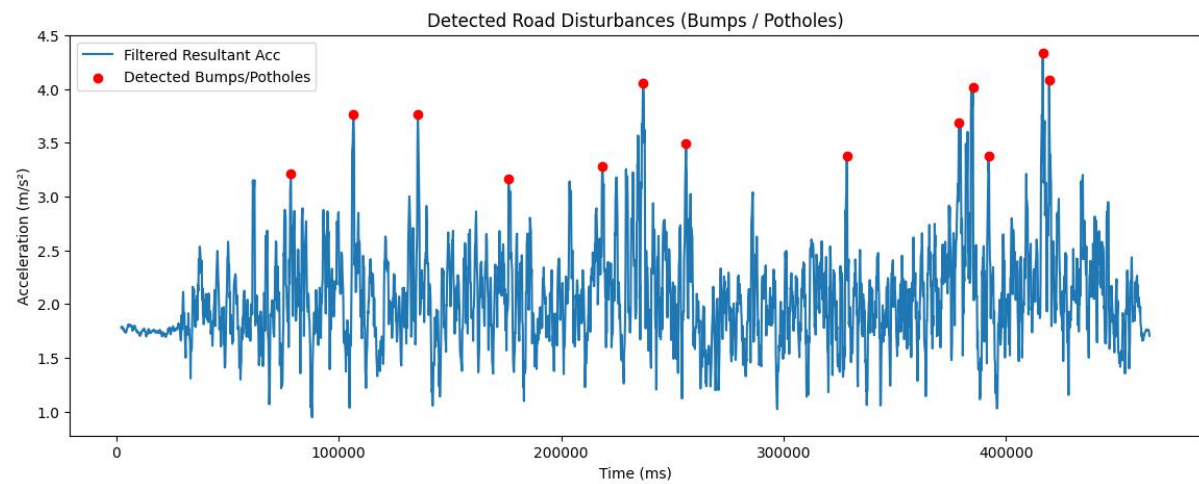
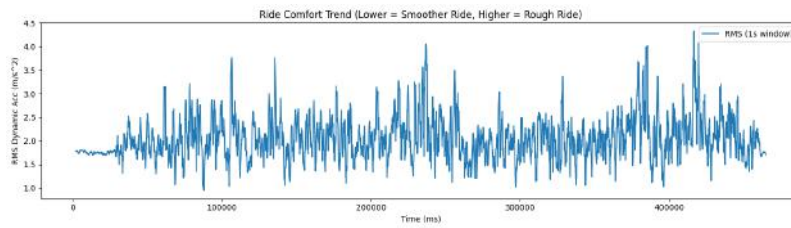
6. Connection Diagram

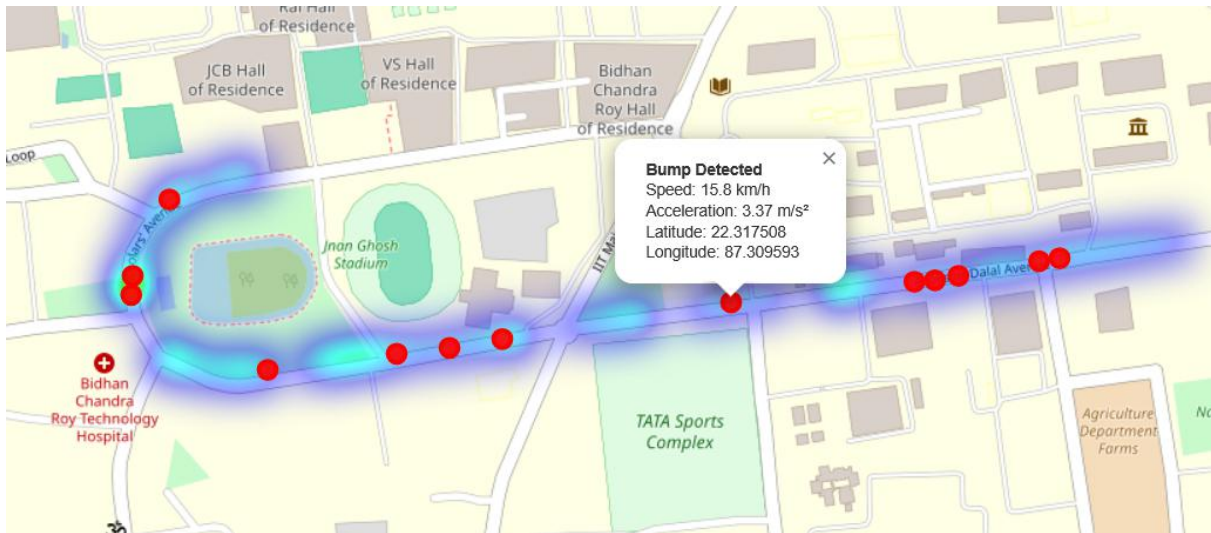


7. Results

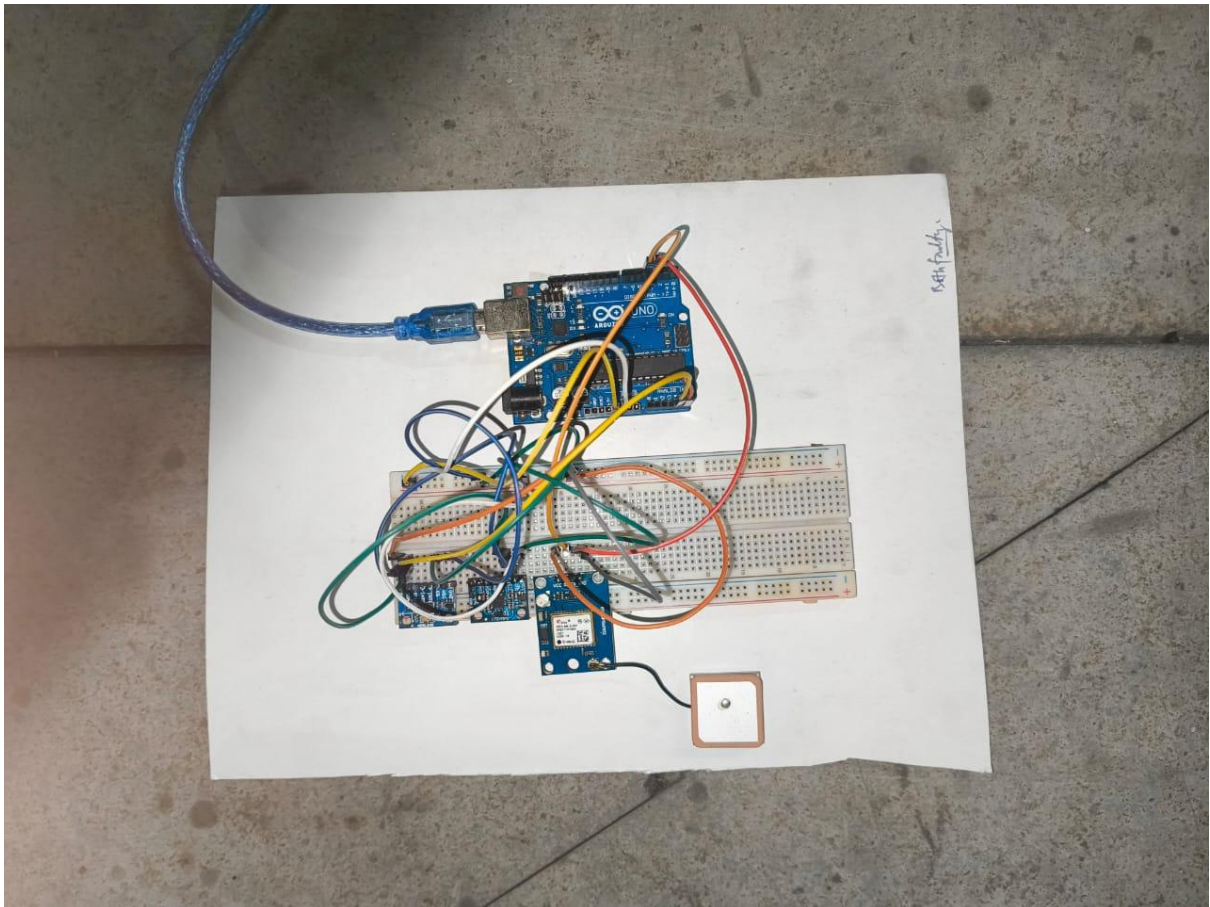
When it comes to the results, these are the major graphs that we obtained.







The red circles indicate bumps of certain prominence.



Our circuit on a cardboard plating.



Circuit double taped to the toto floor and laptop connected. Coolterm used to collect the data.

8. Discussion

- **Vehicle influence:** The Toto's soft suspension increases sensitivity to small bumps — useful for passenger-comfort assessment but implies measured magnitudes are vehicle-dependent. For structural pavement assessment, vehicle-type correction or reference vehicle data may be required.
- **Sensor redundancy:** The use of both MPU6050 and ADXL345 improved confidence in detected events. Slight amplitude differences are expected due to range, filter, and axis alignment.
- **GPS limitations:** Standard NEO-6M GPS at 1 Hz gives spatial resolution $\sim 3\text{--}5$ m at walking speed. For high-precision localization, higher-rate GNSS or differential methods are suggested.
- **Repeatability:** Two passes on the same segment should be compared — high repeatability increases confidence in detected road defects.

9. Conclusions

1. Low-cost MEMS accelerometers (MPU6050 and ADXL345) mounted on a Toto effectively detected and localized road irregularities along the 2.2 Road at IIT Kharagpur.
2. The combined IMU+GPS approach produced a spatial map of road roughness useful for maintenance prioritization.
3. Sensor fusion and proper gravity compensation improved the quality of vibration metrics.
4. Future improvements include higher GPS sampling, more passes for statistical validation, and potentially a fleet-based collection to average vehicle-specific biases.

10. Recommendations

- We have to prioritize repair at high-severity hotspots (table with coordinates).
- For routine monitoring, we can use a standard mounting plate location and sampling frequency.
- We can improve data quality by using GPS with higher update rate or an external antenna for better connection.
- For human comfort assessment, we can also consult ISO 2631-1 thresholds before declaring health risk.
- If such similar experiment is conducted multiple times, at varying speeds, and a parametric sweep of hyperparameters like prominence of bump peaks and others, we can pinpoint exactly where major bumps exist and focus attention there.

11. References

1. Behaviour of Metro Coach on Newly Built Track in Kolkata – Prof. V. Racherla, Dr. S.K. Singh, Dr. H. Danawe.
2. MPU-6050 Product Specification (InvenSense) datasheet.
3. ADXL345 Datasheet (Analog Devices).

12. Appendices

A. Arduino code

```
#include <Wire.h>
#include <MPU6050.h>
#include <TinyGPS++.h>
#include <SoftwareSerial.h>
#include <Adafruit_Sensor.h>
#include <Adafruit_ADXL345_U.h>

MPU6050 mpu;
TinyGPSPlus gps;
SoftwareSerial gpsSerial(3,4);
Adafruit_ADXL345_Unified adxl = Adafruit_ADXL345_Unified();

unsigned long prevTime;
float prevGx = 0, prevGy = 0, prevGz = 0;

void setup() {
```



```

Serial.begin(9600);
gpsSerial.begin(9600);
Wire.begin();
mpu.initialize();

if (!mpu.testConnection()) {
  Serial.println("MPU6050 connection failed!");
  while(1);
}
if (!adxl.begin()) {
  Serial.println("ADXL345 not detected. Check wiring!");
  while(1);
}

Serial.println("Time(ms),Lat,Lon,Speed(kmph),Ax,Ay,Az,Gx,Gy,Gz,AlphaX,AlphaY,AlphaZ");
prevTime = millis();
adxl.setRange(ADXL345_RANGE_2_G);
}

void loop() {

// Serial.print("Satellites: ");
// Serial.println(gps.satellites.value());

while (gpsSerial.available()) {
  gps.encode(gpsSerial.read());
}

float Ax = (mpu.getAccelerationX() / 16384.0) * 9.81;
float Ay = (mpu.getAccelerationY() / 16384.0) * 9.81;
float Az = (mpu.getAccelerationZ() / 16384.0) * 9.81;

float Gx = mpu.getRotationX() / 131.0;
float Gy = mpu.getRotationY() / 131.0;
float Gz = mpu.getRotationZ() / 131.0;

unsigned long currTime = millis();
float dt = (currTime - prevTime) / 1000.0;

float AlphaX = (Gx - prevGx) / dt;
float AlphaY = (Gy - prevGy) / dt;
float AlphaZ = (Gz - prevGz) / dt;

Serial.print(currTime); Serial.print(",");

if (gps.location.isValid()) {
  Serial.print(gps.location.lat(), 6); Serial.print(",");
  Serial.print(gps.location.lng(), 6); Serial.print(",");
} else {
  Serial.print("0,0,");
}

if (gps.speed.isValid()) {
  Serial.print(gps.speed.kmph(), 2); Serial.print(",");
} else {
  Serial.print("0,");
}

Serial.print(Ax); Serial.print(",");
Serial.print(Ay); Serial.print(",");
Serial.print(Az); Serial.print(",");
Serial.print(Gx); Serial.print(",");
Serial.print(Gy); Serial.print(",");
Serial.print(Gz); Serial.print(",");
Serial.print(AlphaX); Serial.print(",");
Serial.print(AlphaY); Serial.print(",");
Serial.print(AlphaZ); Serial.print(",");

prevGx = Gx;
prevGy = Gy;
prevGz = Gz;
prevTime = currTime;

sensors_event_t event;
adxl.getEvent(&event);
float Ax_ADXL = event.acceleration.x;
float Ay_ADXL = event.acceleration.y;
float Az_ADXL = event.acceleration.z;

Serial.print("Ax from ADXL: "); Serial.print(Ax_ADXL, 3);
Serial.print("Ay from ADXL: "); Serial.print(Ay_ADXL, 3);
Serial.print("Az from ADXL: "); Serial.print(Az_ADXL, 3); Serial.println("");

delay(10); // 100 Hz IMU logging
}

```

B. Python code for data analysis

Ride Analysis 2.2 Toto.ipynb